Continuous Deployment as Cost-Sensitive Decision-Making: When Contextual Bandits Outperform Static Rules and When They Don't

Abishek Kumar Giri
Stockton University
giriabishekkumar5@gmail.com
ORCID: 0009-0005-9082-9887

Abstract

Continuous deployment pipelines make deployment decisions millions of times per day across the industry, yet whether to treat them as classification or as sequential cost minimization has not been empirically settled. This paper characterizes the operating envelope of contextual-bandit deployment control: the cost-asymmetry, failure-rate, and trajectory-length conditions under which adaptive policies outperform static rules, and the conditions under which they do not. We apply disjoint LinUCB (Li et al., 2010) and Bayesian linear Thompson Sampling (Agrawal & Goyal, 2013) to the three-action problem {deploy, canary, block} with an asymmetric cost matrix and a pending-reward buffer enforcing the delayed-feedback invariant.

The strongest result is the cost-ratio sweep: the bandit advantage over a static rule scales monotonically from -7% at 5:1 to -49.6% at 100:1. The default 20:1 ratio sits precisely at break-even; the case for adaptive control is material only above 40:1. Within the bandit family, Thompson Sampling outperforms LinUCB by 6.8% on real GitHub Actions CI data (Thompson 624 vs. LinUCB 669.5; bootstrap 95% CI [598, 648] entirely below LinUCB mean, $p < 0.0001$ over 10,000 resamples). The static-rule cost (644.5) falls inside Thompson's CI — the two are statistically tied. In this regime, the exploration-strategy choice matters more than the bandit-vs-rule choice.

The envelope has a hard lower boundary. On real GitHub Actions data (600 runs, two public repositories) with a 5.3% failure rate and no commit-level features, LinUCB over-blocks and costs 3.8% more than a static rule; 300-step trajectories leave the policy below the $O(d^2) = 169$ updates-per-arm convergence threshold. Drift adaptation via Page-Hinkley detection is net-negative across all three drift modes — stationary, abrupt, and gradual — due to false-alarm resets at the default sensitivity ($\lambda = 50$), firing 10.6 resets per trajectory on stationary data. The bandit framing adds genuine value by making cost assumptions explicit and treating the canary decision as a first-class action; the algorithmic gains over static rules are conditional on operating above the envelope boundary.

1. Introduction

1.1 The Deployment Decision Problem

Every passing continuous integration (CI) build triggers a deployment decision: ship to production, route through a canary, or block? A failed production deployment means incidents, on-call pages, and rollbacks. A blocked safe change costs developer time. A canary offers partial protection at throughput cost. Getting this right matters, and at scale it happens thousands of times per day.

Current tooling treats this as a prediction problem. Just-in-time defect prediction models (Kamei et al., 2013; McIntosh & Kamei, 2018) estimate failure probability and apply a static threshold. Three structural limitations follow:

1. **The threshold encodes an implicit cost assumption.** Setting it requires a judgment about the cost of blocking safe changes vs. deploying bad ones. This assumption is rarely explicit and never adapts.
2. **Costs are asymmetric and context-dependent.** A production incident at a payment service costs an order of magnitude more

than one at an internal tool. A single fixed threshold cannot represent this.

3. **The model does not learn from outcomes.** JIT models are trained offline on defect labels. They receive no feedback when predictions lead to incidents or unnecessary blocks.

1.2 Our Framing: Sequential Cost Minimization

We model deployment control as a **contextual bandit**: at each decision step t, the controller observes a context vector x_t (commit metadata, CI signal, change type, author history) and selects an action $a\_t \in$ {deploy, canary, block}. A reward $r\_\{t+k\_t\}$ = *-cost(a_t, outcome_{t+k_t})* arrives k_t steps later, where k_t reflects the time until the deployment outcome is observable.

This framing differs from classification in three ways:

- **The objective is cumulative cost minimization, not accuracy maximization.** The policy is evaluated on what it costs to act, not on whether it correctly predicts a label.
- **The three-action space makes the canary option a first-class decision.** We model the action directly rather than applying a threshold to a risk score.
- **The policy learns online from its own decisions.** Each deployment outcome updates the policy's belief about which actions are cost-effective in which contexts.

1.3 When Does the Bandit Framing Pay Off?

The bandit advantage is not unconditional. Two conditions must hold simultaneously for a contextual bandit to outperform a well-tuned static rule:

Condition 1 — sufficient failure rate asymmetry. When failure rates are very low (say, 5%), the deploy arm is nearly always optimal regardless of context. A static rule that deploys aggressively matches the oracle. A bandit with exploration budget α wastes decisions sampling suboptimal arms (canary, block) before its posterior converges. The exploration cost dominates the expected gain.

Condition 2 — sufficient informative context. LinUCB with a d-dimensional feature vector needs roughly $O(d^2)$ updates per arm before its parameter estimates are statistically meaningful. With $d = 13$ in our setup, that is approximately 169 updates per arm before the confidence interval shrinks to first-order accuracy. On a 300-step real-world trajectory, the bandit has barely enough data to form reliable per-arm estimates, let alone differentiate arms based on context. If the feature vector carries no commit-level signal (no files changed, no test counts), the bandit degenerates to learning from failure rate and a bias term — information a static rule can encode directly.

When both conditions fail — low failure rate, feature sparsity, short trajectory — static rules are competitive and bandits may be worse. This paper documents both regimes empirically.

Beyond the two conditions, our experiments surface an orthogonal limitation of posterior sampling: even after convergence (200 updates per arm for smoke/alpha, 183 for smoke/beta, both above the $O(d^2)$=169 threshold), Thompson Sampling's posterior never collapses to the optimal arm. Thompson persistently allocates 22.4% to canary at a failure rate where block is cheaper, while LinUCB's upper confidence bound (UCB) criterion concentrates on the highest-confidence arm and reaches 86.3% block. Posterior variance, not sample size, is the driver. We discuss this in §5.1.

1.4 Contribution Summary

LinUCB and Thompson Sampling are well-established algorithms. Our contributions are in problem framing and empirical characterization:

- **Reframing:** Deployment control recast as sequential cost minimization with a three-action space {deploy, canary, block} and an explicit asymmetric cost matrix. Cumulative cost replaces accuracy as the primary metric.
- **Evaluation framework:** Online-replay protocol with a pending-reward buffer enforcing the delayed-feedback invariant; explicit bias disclosure; cost as the sole headline metric.
- **Component ablation:** Cost weighting, delayed feedback, and drift adaptation isolated and quantified. Cost weighting is the dominant factor (+31% cumulative cost when removed); drift adaptation, under the chosen Page-Hinkley calibration, is net-negative across all three drift modes due to false-alarm resets — a measured limit of off-the-shelf drift detectors on cost-stream data.
- **Two-sided empirical characterization:** Bandits outperform static rules when failure costs are high and context is informative. In the low-failure regime with feature sparsity (5.3% failure rate, no commit-level features), UCB-based policies over-block and cost 3.8% more than a static rule — a negative result that defines the operating envelope of the framing. A cost-ratio sweep (5:1 to 100:1) confirms the advantage is monotone: the default 20:1 ratio in this paper sits at the envelope boundary where bandits

break even; at 40:1 and above, the advantage is 19–50%.

- **Within-bandits exploration finding:** In the feature-sparse regime where LinUCB underperforms, Thompson Sampling outperforms LinUCB by 6.8% on real data (CI [598, 648] entirely below LinUCB = 669.5; $p < 0.01$). Thompson and the static rule are statistically tied. The choice of exploration strategy — posterior sampling vs. UCB — is the first-order variable in this regime, not the bandit framing itself.

1.5 Related Work

Just-in-time defect prediction. Kamei et al. (2013) establish the empirical foundation for commit-level defect prediction, training logistic regression models on change metrics (churn, complexity, developer experience) across 10,000+ commits from six large open-source projects. They demonstrate that commit-level features predict defect introduction with AUC above 0.70 and that developer experience is among the most informative features. What they do not model: the asymmetric operational cost of prediction errors, the adaptive adjustment of thresholds to project-specific failure distributions, or the three-way deploy/canary/block action space. McIntosh & Kamei (2018) show that fix-inducing changes are a moving target — the feature importance and model accuracy of JIT prediction shift substantially across time periods — motivating approaches that adapt online rather than training a fixed offline model.

Contextual bandits. Li et al. (2010) formulate personalized news recommendation as a contextual bandit problem and introduce LinUCB, demonstrating that upper-confidence-bound policies using linear payoff models outperform static and context-free baselines in online A/B comparisons. They do not address asymmetric costs, delayed rewards, or three-way action spaces. Chu et al. (2011) provide theoretical regret analysis for linear contextual bandits, establishing that the cumulative regret of LinUCB scales as $O(\sqrt{(dT \log T)})$ where d is the feature dimension and T is the horizon; this bound underlies the $O(d^2)$ convergence threshold used in §1.3.

Evaluation under partial feedback. Joachims et al. (2018) address learning from logged bandit feedback, where only the reward for the chosen action is observed. They introduce propensity-weighted empirical risk minimization to debias learning from such data and show that naive learning from logged feedback produces systematically biased models. Their analysis is relevant to the online replay evaluation used here: because every logged action in the Travis-Torrent data is DEPLOY, inverse propensity scoring cannot be applied (log propensity is identically 1.0), and the bias they identify is present in our results without correction. We disclose this explicitly in §7.1.

Concept drift. Gama et al. (2014) survey concept drift adaptation algorithms for data streams, covering drift detectors, ensemble methods, and sliding-window retraining. They characterize the trade-off between detection speed and false-alarm rate that motivates threshold calibration — the same trade-off that produces the false-alarm problem in §6.4. Bifet & Gavaldà (2007) introduce ADWIN, an adaptive windowing algorithm that detects changes in the mean of a stream statistic by testing whether older and newer subwindows have significantly different distributions. ADWIN provides theoretical false-alarm and detection-delay guarantees that Page-Hinkley lacks; it is implemented in this codebase but excluded from the reported experiments because it was not evaluated on the cost streams used here.

Delayed feedback in bandits. Vernade et al. (2017) study the stochastic bandit problem where rewards arrive after a random delay and may be censored (never arrive). They show that naive algorithms ignoring delay structure suffer avoidable regret, and they propose algorithms with regret bounds that account for delay distribution. Their setting maps directly to the deployment evaluation problem: a CI run takes minutes to hours, and its deployment outcome (incident or not) may not be observable within the evaluation window. Joulani et al. (2013) address the same delayed-feedback setting for general online learning, showing that the regret cost of delay is $O(\sqrt{(d\_max \cdot T)})$ where d_max is the maximum delay. Pike-Burke et al. (2018) extend delay analysis to aggregated anonymous feedback. This paper applies the pending-reward buffer approach from Joulani et al. (2013) to the deployment context, routing all policy updates through a buffer that enforces the delayed-feedback invariant.

---

2. Problem Formulation

2.1 Contextual Bandit with Delayed Rewards

Let X be the space of pre-action observable features, $A = \{\text{deploy, canary, block}\}$, and $O = \{\text{success, failure, censored, blocked}\}$ the outcome space.

At step t the agent observes $x\_t \in X$, selects $a\_t \in A$, and receives a reward

```
r_{t+k_t} = -cost(a_t, outcome_{t+k_t})
```

after a delay of k_t steps. In our experiments, $k\_t = max(1, \lceil duration\_t / 60 \rceil)$ steps, where *duration_t*

is the build duration in seconds. The pending-reward buffer holds the reward until step $t + k\_t$; the policy has no information about it before that step.

When the outcome is not yet resolved by step t + *max_delay*, the reward is **censored** and excluded from policy updates.

2.2 Context Features

The feature vector $x\_t \in \mathbb{R}^{13}$ encodes 12 pre-action observables (normalised) plus a bias term:

All features are available before the deployment action. No post-deploy signal or outcome appears in x_t.

2.3 Cost Matrix

The cost function *cost:* $A \times O \to \mathbb{R}_{\geq 0}$ encodes operational priorities:

Cost asymmetry is deliberate: deploy + failure (10) is 20× block + would_fail (0.5). Reward is r = *-cost*. The cost matrix is configurable; §4.3 sweeps it.

2.4 Objective

The policy π*:* $X \to A$ minimizes cumulative operational cost over T decisions:

```
J($\pi$) = $\Sigma$_{t=1}^{T} cost(a_t, outcome_t, t)
```

Cumulative operational cost is the sole primary metric throughout this paper. Action distributions (deploy%, canary%, block%) are reported as diagnostics to explain *why* a cost difference arose.

---

3. Method

3.1 LinUCBWithDrift (Disjoint LinUCB with Cost-Sensitive Reward and Optional Drift Reset)

`LinUCBWithDrift` applies disjoint LinUCB (Li et al., 2010; Chu et al., 2011) to the deployment decision, replacing click-through reward with negative operational cost and routing all updates through the delayed-feedback buffer. Its update equations are identical to plain LinUCB; the only structural addition is an optional PageHinkley detector that can reset the per-arm weight matrices on drift events.

Per-arm model. For each arm $a \in A$:

```
A_a $\in$ $\mathbb{R}$^{d$\times$d},  initialized to $\lambda$I
b_a $\in$ $\mathbb{R}$^d,       initialized to 0
$\theta$_a = A_a^{-1} b_a
```

Action selection:

```
UCB(a) = $\theta$_a^T x_t + $\alpha$ $\sqrt{}$(x_t^T A_a^{-1} x_t)
a_t    = argmax_{a $\in$ A} UCB(a)
```

Update rule (on matured reward *(x_i, a_i, r_i)*):

```
A_{a_i} $\leftarrow$ A_{a_i} + x_i x_i^T
b_{a_i} $\leftarrow$ b_{a_i} + r_i x_i,  where  r_i = -cos
```

The negative cost signal means the policy learns to prefer arms with lower expected operational cost.

3.2 Thompson Sampling Baseline

Bayesian linear Thompson Sampling (Agrawal & Goyal, 2013) maintains a per-arm Gaussian posterior over weight vectors:

```
Prior:     $\theta$_a ~ N(0, v$_{0}$ $\cdot$ I)
Posterior: $\Lambda$_a = (1/v$_{0}$)I + (1/$\sigma$$^{2}$)
           b_a = (1/$\sigma$$^{2}$) $\Sigma$ r$_{i}$ x$_{
           $\mu$_a = $\Lambda$_a^{-1} b_a
```

Action selection samples $\theta\_a \sim N(\mu\_a, \Lambda\_a^{-1})$ via Cholesky decomposition and picks *argmax_a* $\theta\_a^\top$ x_t. Exploration is implicit; no α parameter is needed. Default hyperparameters: v_0 = *1.0*, σ^2 = *0.1*. Updates use the same negative-cost reward as `CostSensitiveBandit`.

Thompson's stochastic action selection produces nonzero seed variance: identical data yields different action choices across seeds. This makes it the only policy for which bootstrap CIs are informative in our current experiment setup (all other policies are deterministic given the same delayed-feedback schedule).

3.3 Delayed Reward Buffer

The `PendingRewardBuffer` (`delayed/buffer.py`) holds *(context, action, cost, outcome, censored)* until `pop_available(t + k_t)` is called. No model update occurs before maturity.

The buffer's value is correctness, not performance. Removing it improves cost by 1.1% (§5.3) because immediate feedback accelerates convergence. The buffer exists to prevent a policy from using future information at decision time — a requirement for valid evaluation in any real deployment setting.

3.4 Drift Adaptation (Exploratory — Excluded from Main Claims)

`CostSensitiveBandit` optionally accepts a PageHinkley detector (Mouss et al., 2004) that triggers model reset on detected distribution shift (Gama et al., 2014). On stationary data, the detector at λ_PH = *50* fires 44 false alarms over 1,150 steps, making the full model 27%-more expensive than the no-drift variant. Drift results are excluded from main claims; the threshold requires calibration against non-stationary data not yet available.

Dimension	Feature	Normalisation
0	files_changed	÷ 50
1	lines_added	÷ 1000
2	lines_deleted	÷ 500
3	src_churn	÷ 1500
4	is_pr	binary
5	tests_run	÷ 200
6	tests_added	÷ 10
7	build_duration_s	÷ 180
8	author_experience	÷ 10
9	recent_failure_rate	[0, 1]
10	has_dependency_change	binary
11	has_risky_path_change	binary
12	bias	1.0

Action	Outcome	Cost	Interpretation
deploy	success	0.0	Successful rollout — no cost
deploy	failure	10.0	Production incident; on-call, rollback
canary	success	1.0	Canary overhead + promotion latency
canary	failure	4.0	Partial-rollout incident, limited blast
block	would succeed	2.0	Safe change delayed; developer wait
block	would fail	0.5	Risky change correctly held back
block	unknown	2.0	Counterfactual unobserved (replay)

Policy	Description	Learning
`static_rules`	Deterministic threshold rule (files changed, failure rate, risky paths)	None
`heuristic_score`	Weighted risk score → thresholded action; fixed weights	None
`linucb`	Disjoint LinUCB (Li et al., 2010), r = *-cost*	UCB exploration
`thompson`	Bayesian linear TS (Agrawal & Goyal, 2013), r = *-cost*	Posterior sampling

3.5 Baselines

At identical α, λ, and reward signal, `linucb` and `cost_sensitive_bandit` are mathematically identical ($\|$b-vector difference$\|_2 = 0$ confirmed experimentally). They are reported together in the main table.

4. Experiments

4.1 Synthetic Dataset

The primary experiment uses a synthetic dataset (`data/raw/travistorrent_smoke.csv`) generated to match the TravisTorrent schema (Beller et al., 2017):

Limitations: Two projects, fixed failure rates, deterministic delay model. Bootstrap CIs collapse to zero for all deterministic policies. Thompson Sampling is the only policy with nonzero seed variance. No cross-project generalization is possible from this dataset.

4.2 Online Replay Setup and Bias

We use online replay (`evaluation/online_replay.py`): policies learn during trajectory traversal. Costs are *cost(policy_action, logged_CI_outcome)*, using CI outcome as a counterfactual proxy for the deployment outcome.

Online replay is a biased evaluation. Every logged action is DEPLOY (the loader hardcodes this). Costs for CANARY and BLOCK are counterfactual: we assume CI outcome would be the same regardless of deployment action. This is approximately valid for TravisTorrent (CI runs before any deployment) but unverifiable in general. We use online replay exclusively for learning-dynamics verification, not causal cost estimation.

Configuration (synthetic experiment):

4.3 Robustness Conditions

Five conditions test whether conclusions depend on cost matrix or delay setting:

Percentile bootstrap CI: 10,000 resamples, seed 42, over seeds 0–4.

4.4 Ablation Conditions

4.5 Real-World Sanity Check Dataset

To test whether the synthetic findings are contradicted by real CI data, we collected GitHub Actions workflow run history from two public repositories via the GitHub REST API:

Schema mapping: `head_sha` → `git_trigger_commit`, `conclusion` → `tr_status`, timestamps → duration. No commit-level features are available without additional per-commit API calls; fields default to 0. The feature vector is

Project	Builds	Failure rate	History span
`smoke/alpha`	600	15%	>365 days
`smoke/beta`	550	35%	>365 days

Parameter	Value
α	1.0
λ	1.0
Delay model	*k_t = max(1, ⌈build_duration_s / 60⌉)*
Cost matrix	Default (§2.3)
Seeds	0–4
flush_at_end	True

Condition	Key change
Default	—
High failure	deploy_failure: 10→20, canary_failure: 4→8
Low block	block_safe: 2→1, block_unknown: 2→1
Short delay	delay_step_seconds: 60→120
Long delay	delay_step_seconds: 60→30

Variant	What changes
`no_drift` (reference)	No drift reset
`no_delay`	Immediate updates (no buffer)
`no_cost`	Binary reward (-1/0 instead of -cost)
`full`	+ PageHinkley drift reset

dominated by the bias term and recent-failure-rate — 11 of 13 dimensions carry no project-specific signal.

Filter relaxation: Standard filters (`min_builds=500`, `min_history_days=365`) cannot be satisfied by a 20–99 day API sample. Relaxed to `min_builds=100`, `min_history_days=0`. Results are not directly comparable to the synthetic conditions.

Censored rewards: Cancelled CI runs (≈3%) produce genuine censored rewards — the first experiment confirming real-world censoring rather than theoretical.

5. Results

5.1 Main Results (Synthetic Data)

[Preliminary — synthetic 2-project dataset. Deterministic policies have zero CI width.]

Figure 1 (`fig_cumulative_cost_synthetic`): Cumulative cost per policy on the synthetic dataset (n=30 seeds, error bars = ±1 std). Thompson's higher cost and wider spread relative to LinUCB and static rules are visible; see also Figure 6 for the seed distribution.

Figure 2 (`fig_cumulative_cost_curves`): Cumulative cost over time (mean across 30 seeds), synthetic dataset. All deterministic policies follow identical step functions; Thompson shows seed-driven variance.

Table 1: Cumulative cost, default cost matrix, synthetic dataset. All 1,150 rewards matured (0 censored). Thompson: mean ± std across seeds 0–29 (n=30), bootstrap seed 42. *Takeaway: Thompson is 1.4% more expensive than static on aggregate (CI [1884, 1922]; static = 1878). The 5-seed result that showed Thompson as cheapest was a sampling artifact: three of the five seeds hit low-cost trajectories below the population mean. The n=30 result reverses the direction with a non-overlapping CI.*

`linucb` and `cost_sensitive_bandit` are identical at the same α and λ — mathematically expected ($\|b\text{-vector difference}\|_2 = 0$).

The aggregate tie (1878 vs. 1879) is a dataset artifact. **Per-project, the bandit and static rule take opposite positions:**

At 15% failure, canary is cheaper than block (1.45 vs. 1.775/step); the static rule's canary-heavy strategy is near-optimal and LinUCB over-blocks. At 35% failure, block is cheapest (1.475/step); LinUCB correctly converges to 86.3% block.

Thompson Sampling costs 1.4% more than static rules in aggregate across n=30 seeds (1903 vs 1878, CI [1884, 1922]; the CI does not overlap static=1878). The n=5 mean of 1877 was a sampling artifact: three of the five seeds fell below the population mean, reversing the sign. With n=30, std = 54 (range 1814–2010); action distribution: 22.4% canary vs. 5.7% for LinUCB, reflecting higher posterior uncertainty. On the synthetic dataset, Thompson is

Project	Runs	Failure rate	Span
`psf/requests`	300	5.3%	20 days
`pallets/flask`	300	23.3%	99 days

Policy	
`static_rules`	1150
`heuristic_score`	1150
`linucb` / `cost_sensitive_bandit`	1150
`thompson` (n=30 seeds)	1150

Project	Failure rate
`smoke/alpha`	15%
`smoke/beta`	35%

1.4% more expensive than the static rule (1903 vs 1878, n=30, CI [1884, 1922] non-overlapping). The mechanism is not a Condition 2 failure — trajectories of 600 and 550 steps clear the $O(d^2)$=169 threshold with 200 and 183 updates per arm. Instead, Thompson's posterior never fully collapses: it allocates 22.4% to canary even at failure rates where block is the optimal arm, while LinUCB reaches 86.3% block on the same data. The cost penalty is structural to posterior sampling rather than a data-scarcity artefact (see §1.3 supplementary observation).

5.2 Robustness Results (Synthetic Data)

[Preliminary — synthetic data. Bootstrap CIs are zero-width for deterministic policies.]

Cost matrix sweep:

Under high failure cost (deploy_failure=20): the bandit outperforms static rules by **485 units (19%)** by shifting to 92.7% block. Static rules cannot adapt their fixed thresholds. Under low block penalty (block_safe=1): the bandit saves **420 units (27%)** by exploiting cheaper blocking.

Delay sweep:

Under long delay (doubled step-count delay), the bandit's uninformed-prior phase extends and costs 19 units more than static rules (1.0%), consistent with $O(\sqrt{(d\_max \cdot T)})$ delay-induced regret inflation (Pike-Burke et al., 2018). Under short delay, the bandit gains 29 units by converging faster.

5.3 Ablation Study (Synthetic Data)

[Preliminary — synthetic data. All 5 seeds identical for deterministic variants.]

Figure 3 (`fig_ablation_bars`): Bar chart comparing cumulative cost across the four ablation variants. `no_cost` and `full` are dramatically more expensive than `no_drift` and `no_delay`, isolating cost weighting as the dominant factor.

Table 2: Component ablation, default cost matrix, seed 0. Reference = `no_drift`. *Takeaway: cost weighting is the dominant component (+31%); drift detection is destructive on stationary data (+27%); the buffer costs 1.1% for temporal validity.*

Cost weighting (+31%) is the dominant component. Binary reward (-1/0) gives the block arm the same signal as deploy for failures it correctly avoided — wrong gradient. The 31% degradation quantifies the cost of ignoring the asymmetric cost matrix.

Delay removal (-1.1%). Removing the buffer improves performance slightly because immediate feedback accelerates convergence. The buffer's purpose is temporal validity, not performance. The 1.1% is the honest price of a correct evaluation.

Drift on stationary data (+27%). PageHinkley at λ_PH = 50 fires 44 false alarms, resetting learned weights each time and reverting to deploy-heavy uninformed priors (40.4% deploy rate post-reset vs. 8.0% converged). This is detector behavior on stationary data; it does not characterize the component's value on non-stationary data. Drift results are **excluded from main claims**.

5.4 Real-World Sanity Check

[Highly preliminary — real GitHub Actions data, 2 projects, feature sparsity, relaxed filters, biased evaluation. Do not compare magnitudes against synthetic results.]

Figure 4 (`fig_cumulative_cost_real`): Cumulative cost per policy on real GitHub Actions data (n=30 seeds, error bars = ±1 std). Thompson's lower mean and high variance relative to LinUCB are visible; static rules sit between the two bandit policies.

Figure 5 (`fig_action_distribution`): Stacked-bar action distribution (deploy/canary/block) for all policies on synthetic (left) and real (right) datasets. LinUCB's block-heavy convergence and Thompson's persistent canary allocation are visible on both panels.

Policy	Default (df=10)	High failure (df=20)	Low block (bs=1)
`static_rules`	1878	2564	1584
`heuristic_score`	2319	4103	2319
`linucb`	1879	**2079**	**1164**
`cost_sensitive_bandit`	1879	**2079**	**1164**

Policy	Default (60s)	Short delay (120s)	Long delay (30s)
`static_rules`	1878	1878	1878
`linucb`	1879	**1849**	1897
`cost_sensitive_bandit`	1879	**1849**	1897

Variant	
`no_cost` (binary reward)	2469.0
`full` (+ PageHinkley drift)	2383.0
`no_drift` (reference)	1879.0
`no_delay` (immediate updates)	**1857.5**

Component	
Cost weighting → binary (`no_cost`)	+590
Drift reset on stationary data (`full`)	+504
Delayed buffer → immediate (`no_delay`)	-21.5

Figure 6 (`fig_thompson_seed_distribution`): Boxplot of Thompson's per-seed cumulative cost distribution across 30 seeds, synthetic (left) and real (right). The real-data spread (std=72, range 445.5–725.5) illustrates that no individual seed reliably dominates static rules.

Figure 7 (`fig_cost_cdf_per_step`): Empirical CDF of per-step costs pooled across all seeds and steps. The heavier left tail for Thompson on real data reflects its lower block rate and correspondingly more deploys on low-failure projects.

Table 3: Online replay on real GitHub Actions data, default cost matrix, seeds 0–29 (n=30), bootstrap seed 42. *Takeaway: LinUCB over-blocks and costs 3.8% more than static; Thompson outperforms LinUCB by 6.8% but is statistically tied with static (static=644.5 falls inside Thompson's CI [598, 648]).*

Thompson per-seed range: 445.5–725.5 (seeds 0–29); 95% CI [598, 648].

Per-project expected cost analysis:

Finding R1 (negative result): LinUCB over-blocks in the low-failure regime and costs more than static rules (669.5 vs. 644.5). At 5.3% failure, deploy is optimal (0.53/step); blocking costs 1.92/step. LinUCB converges to 42.2% block across both projects because feature sparsity prevents differentiation — both projects look similar with zero files_changed, zero tests_run, etc. The bandit learns primarily from the recent-failure-rate feature, which does not converge quickly enough to assign different strategies to the two projects within 300 steps. The static rule's explicit threshold on `files_changed` and `recent_failure_rate` happens to produce a 64.3% deploy rate that is close to optimal for this failure-rate distribution.

This finding directly illustrates §1.3 Conditions 1 and 2: the low-failure regime and feature sparsity together prevent the bandit from outperforming a rule that has those conditions hardcoded.

Finding R2: Within the bandit family, Thompson outperforms LinUCB by 6.8%; both are statistically tied with static. Among bandit policies, Thompson Sampling outperforms LinUCB by 6.8% on real data (Thompson 624, LinUCB 669.5; CI [598, 648] entirely below LinUCB; $p < 0.01$ paired bootstrap). Thompson and the static rule are statistically tied: static=644.5 falls inside Thompson's CI. The exploration-strategy choice — posterior sampling versus upper confidence bound — matters more in this regime than the bandit-vs-rule choice. Posterior variance is high (std=72, range 445.5–725.5 across seeds 0–29): on 300-step trajectories, Thompson's posterior has not converged and individual seeds swing from 31% below static (seed 0: 445.5) to 12.5% above (seed 28: 725.5).

Finding R3: Real-world censoring is present. Cancelled CI runs produce 17–22 censored rewards per trajectory (≈3%), varying by policy. This confirms reward censoring is not merely theoretical.

Policy	
`static_rules`	600
`heuristic_score`	600
`linucb` / `cost_sensitive_bandit`	600
`thompson` (n=30 seeds)	600

Project	Failure rate	E[deploy]	E[canary]	E[block]	Optimal
`psf/requests`	5.3%	**0.53**	1.16	1.92	deploy
`pallets/flask`	23.3%	2.33	1.70	**1.65**	block

6. Key Findings

6.1 Reliable Findings (mechanism verification — hold regardless of dataset)

F1: Cost weighting is the most important model component (+31% cost when removed). Binary reward destroys the signal asymmetry the cost matrix encodes. This is a mechanism result: it holds whenever the cost matrix is asymmetric (20× ratio between deploy+failure and block+would_fail). It does not require real-world validation to hold.

F2: The delayed-feedback buffer enforces correctness at a 1.1% performance cost. Removing the buffer improves performance because immediate feedback accelerates convergence. The buffer's value is validity — preventing use of future information at decision time (Joulani et al., 2013). The 1.1% is the honest cost of temporal correctness.

F3: Page-Hinkley at λ_PH = 50 fires 44 false alarms on 1,150 stationary steps. Expected behavior for a sensitive detector on data with no drift. The threshold requires calibration; we make no claim about its performance on non-stationary data.

F4: Thompson Sampling produces nonzero seed variance (std = 54 synthetic, std = 72 real, n=30 seeds); LinUCB does not. Stochastic posterior sampling yields different per-trajectory behavior. On real short-trajectory data, this variance is large enough that any individual seed can be best or worst of all policies. The tighter n=30 std (54 vs the n=5 pilot value of 72 for synthetic; 72 vs 99 for real) illustrates that five-seed bootstrap CIs were not sufficient to characterize the population mean.

6.2 Preliminary Findings (synthetic data — cannot generalize)

F5: Bandit advantage scales with failure cost severity (+19% at df=20, +27% at low block cost). When the cost matrix moves away from the default implicit assumptions of a fixed static rule, the adaptive policy exploits the new cost structure and the static rule cannot. The qualitative pattern is expected from first principles; the magnitudes are dataset-specific.

F6: Under severe delay, the learning bandit costs 1% more than static rules. Consistent with delay-induced regret inflation. The effect is small and may not survive on real stochastic-delay data.

F7: The aggregate bandit/static tie is a cancellation artifact. At the project level, the bandit wins at 35% failure and loses at 15% failure. The tie is specific to this two-project synthetic dataset design.

6.3 Real-Data Findings (highly preliminary — 2 real projects, feature sparsity)

F8 (negative result): LinUCB over-blocks in the low-failure regime and costs 3.8% more than static rules. On psf/requests (5.3% failure, feature sparsity, 300 steps), the bandit cannot distinguish the project from pallets/flask using the available feature signal. It converges to a block-heavy strategy that is near-optimal for flask but expensive for requests. Static rules' explicit threshold happens to match the optimal strategy for this failure-rate mix. This is precisely the failure mode predicted by §1.3.

F9: Within-bandits exploration finding — Thompson outperforms LinUCB by 6.8% on real data; Thompson and static rules are statistically tied. Among bandit policies, Thompson Sampling outperforms LinUCB by 6.8% on real data (Thompson 624.2, LinUCB 669.5, CI [598, 648] entirely below LinUCB, $p < 0.01$). Thompson and the static rule are statistically tied (the static value 644.5 falls inside Thompson's CI). The exploration-strategy choice — posterior sampling versus upper confidence bound — matters more in this regime than the bandit-versus-rule choice. This result must be read with the convergence caveat of F4: the n=30 point estimate is stable, but no individual seed reliably beats static rules, and the gap will depend on trajectory length and feature signal in other datasets.

F10: Reward censoring is empirically confirmed in real CI data. Cancelled runs produce 3% censored rewards, with policy-dependent variation.

This validates the buffer's censoring path as exercised on real data.

6.4 Operating Envelope (cost-ratio sweep and drift-mode evaluation)

[Synthetic environment / online-replay simulation. These findings characterize the regime where bandits are worth deploying; they do not constitute causal claims about real-world cost savings.]

F11: LinUCB advantage over static rules grows monotonically with the deploy_failure/block_bad cost ratio.

Figure 8 (`fig_cost_sweep`): Line chart of cumulative cost vs. cost ratio (log-x) for static rules, LinUCB, and heuristic policies. LinUCB's advantage widens monotonically; the crossover from parity to advantage occurs near 30:1.

Sweeping five cost-ratio levels (30 seeds each, synthetic dataset):

At the default 20:1 ratio the policies are indistinguishable (difference < 1 unit). As the deploy-failure penalty grows relative to the block cost, LinUCB's ability to learn a block-heavy strategy pays off dramatically — 49.6% cheaper than static rules at 100:1. The relationship is monotone: every doubling of the ratio increases LinUCB's relative advantage. The operating envelope for bandit deployment is therefore high cost-asymmetry regimes (ratio $\geq$ 40:1), where the exploration cost is amortized over a larger penalty gap.

F12: Page-Hinkley drift resets (λ=50) increase cost in all three drift conditions; the no-reset variant matches LinUCB exactly.

Figure 9 (`fig_drift_mode_bars`): Grouped bar chart of mean cumulative cost by drift mode $\times$ policy. `linucb_with_drift_full` is uniformly worst; `linucb_with_drift_no_reset` is visually indistinguishable from `linucb`.

Figure 10 (`fig_drift_recovery_curves`): Cumulative regret over time for all three drift modes (3-panel). Under abrupt and gradual drift, `linucb_with_drift_full` accumulates regret spikes at each reset; `linucb` and `linucb_with_drift_no_reset` track together throughout.

Evaluating six policies across three drift schedules (30 seeds, 500-step synthetic trajectories):

`linucb_with_drift_no_reset` (PageHinkley detector present but resets disabled) is numerically identical to `linucb` in all three modes — confirming the detector is inactive when reset_on_drift=False. `linucb_with_drift_full` (resets enabled) is worse in every condition: 10.6 resets/trajectory under stationarity (false alarms), rising to 25.3 under abrupt drift and 42.2 under gradual drift. The PageHinkley threshold λ=50 is insufficiently conservative for 500-step trajectories with the cost stream's natural variance; it fires on distributional noise rather than genuine concept drift.

Static rules are cheapest under abrupt and gradual drift. This is a boundary effect: static rules do not explore, so they pay no reset cost and have no parameter to unlearn. The bandit's re-learning overhead after a midpoint shift exceeds the gain from adaptation over the remaining 250 steps. Whether longer trajectories would amortize the reset overhead is an open question we do not address here.

Operating envelope summary. The bandit framing is worth deploying under three conditions, each specific to this synthetic setting: cost asymmetry $\geq$ 40:1 (the threshold shifts with failure rate, trajectory length, and feature signal); deployment trajectories long enough that re-learning overhead after drift is amortized; and drift detection calibrated to the cost stream's variance rather than set to a package default. The current paper's 20:1 ratio and 500-step trajectories sit at the boundary of the first condition. That boundary explains the mixed results.

---

7. Threats to Validity and Limitations

7.1 Online Replay is Biased — All Results Are Simulations

Online replay computes costs as *cost(policy_action, logged_CI_outcome)*. Every logged action is DEPLOY. Costs for CANARY and BLOCK are counterfactual, resting on the assumption that CI outcome is independent of the deployment action taken. This assumption is unverifiable. The evaluation measures relative policy rankings within the simulation; it does not measure real-world operational cost.

No logging propensities exist. Inverse propensity scoring cannot be applied — the IPS weight is identically 1.0, reducing the estimator to the direct method. This is unbiased only if the true logging policy always deployed, which is false for human-operated pipelines.

The evaluation is internally consistent (the same counterfactual assumption applies to every policy equally), so relative rankings are meaningful. Absolute cost numbers are simulation artifacts.

Ratio	static_rules	linucb	Δ (LinUCB vs static)
5:1 (df=5, bb=1)	1598	1486	-7.0%
10:1 (df=5, bb=0.5)	1535	1440	-6.2%
20:1 default	1878	1879	+0.05% (tied)
40:1 (df=20, bb=0.5)	2564	2079	-18.9%
100:1 (df=50, bb=0.5)	4622	2330	-49.6%

Drift mode	
none (stationary)	536.9
abrupt (midpoint)	602.7
gradual (25-segment)	685.8

7.2 Synthetic Dataset: Two Projects, Fixed Failure Rates, Zero CI Width

The primary experiment uses 1,150 rows across two synthetic projects at fixed failure rates (15% and 35%). The aggregate tie between LinUCB and static rules (1878 vs. 1879) is a design artifact — the two projects cancel. Deterministic delay model produces zero bootstrap CI width for all policies except Thompson. No claim generalizes beyond this dataset without real multi-project replication.

7.3 Real Sanity Check: Feature Sparsity, Short Trajectories, Relaxed Filters

The GitHub Actions experiment uses a feature vector in which 11 of 13 dimensions carry no commit-level signal. The bandit degenerates to learning from failure rate and bias term. With 300 steps per project and d=13 dimensions, the bandit is near its minimum convergence threshold ($O(d^2) = 169$ updates per arm). Both conditions in §1.3 are violated: psf/requests is in the low-failure regime AND feature sparsity prevents informative context. The negative result (LinUCB loses to static rules) is expected under these conditions and should not be interpreted as evidence that bandits generally underperform static rules. Additionally, the experiment covers only two projects with failure rates of 5.3% and 23.3% — a specific rate combination that happens to favor static rules' explicit threshold. No generalization to the distribution of real-world project failure rates is intended; both the direction and magnitude of the real-data results are specific to this pair.

7.4 LinUCBWithDrift with reset_on_drift=False Is Numerically Identical to LinUCB

$\|$b-vector difference$\|_2 = 0$ at α=1.0, λ=1.0, reset_on_drift=False. The `linucb_with_drift_no_reset` variant matches `linucb` exactly in all three drift-mode conditions. The policies diverge only when the drift detector fires and reset_on_drift=True — i.e., only in the `linucb_with_drift_full` configuration. This confirms `LinUCBWithDrift` is not a distinct algorithm; it is LinUCB plus a drift-triggered weight reset.

7.5 Drift Detection Conclusions Are Conditional on PageHinkley Calibration

Page-Hinkley at λ_PH = 50 fires 44 false alarms on 1,150 stationary steps in the ablation experiment, and 10.6 resets per 500-step trajectory in the drift-mode evaluation. The threshold was set by the Page-Hinkley default and never tuned to the cost stream's variance. All findings about drift adaptation performance — specifically that `linucb_with_drift_full` is 8–28% more expensive than plain LinUCB across all three drift modes — are conditional on this calibration. A lower λ_PH (fewer false alarms) or a different detector (ADWIN; Bifet & Gavaldà, 2007) may change the direction of the drift finding. We report the result for λ_PH = 50 as a measured limit of this detector on this cost stream; we make no claim about whether drift adaptation is intrinsically harmful.

7.6 Thompson Sampling Propensities Are Intractable

The true selection probability for Thompson Sampling requires integrating over the posterior — intractable in closed form. We report propensity = 1.0 throughout. IPS correction cannot be applied to Thompson results even in principle. Thompson's cost estimates carry the standard replay bias plus this additional limitation.

7.7 Seed Counts Below 30 Produce Misleading Direction Findings

Seed counts below approximately 30 produced misleading direction findings in our pilot runs (n=5 showed Thompson as the cheapest policy on synthetic data; n=30 reverses the direction with a non-overlapping CI). All headline numbers in this paper use ≥30 seeds and bootstrap seed 42.

7.8 CI Outcome Is a Proxy for Deployment Failure, Not a Direct Measure

The entire evaluation chain — cost computation, policy update, and result reporting — treats the logged CI outcome (pass/fail) as a proxy for the deployment failure outcome a human engineer would observe in production. This proxy relationship is imperfect in both directions. A CI pass does not guarantee safe deployment: integration-test gaps, environment-specific faults, and configuration drift can produce production incidents on passing builds. A CI failure does not guarantee a production incident would have occurred: flaky tests, dependency version mismatches, and transient infrastructure errors produce false CI failures. The cost model therefore mis-prices a fraction of deployment decisions. The direction and magnitude of this mis-pricing depend on the test suite quality and build environment stability of each repository, neither of which is measured here. All cost claims should be read as costs under the CI-outcome proxy, not costs under the true deployment-failure outcome.

7.9 Real-Data Evaluation Assumes Stationarity; No Drift Analysis Was Performed

The GitHub Actions evaluation treats each 300-step project trajectory as a stationary distribution and uses all steps for both learning and evaluation. If the true CI failure rate changes over the 300-step window — for example, due to a large dependency upgrade, a project refactor, or infrastructure migration — then the earlier steps of the trajectory correspond to a different distribution than the later steps. In that case, a policy that learns from early steps may be mis-calibrated for later steps, producing an underestimate of the cost gap between adaptive and non-adaptive policies. We performed no stationarity test on the real-data trajectories. The drift evaluation in §6.4 is conducted exclusively on synthetic data; its findings do not imply anything about whether the real GitHub Actions trajectories are stationary.

7.10 Synthetic Failure Rates Are Unobservable Hidden State

The synthetic dataset assigns failure rates at the project level (smoke/alpha: 15%, smoke/beta: 35%), but these rates are not part of the feature vector presented to the policy. From the policy's perspective, the failure rate is a latent variable that must be inferred from the reward history. This creates a structural advantage for policies that converge quickly on the majority class: after seeing enough failures, LinUCB correctly assigns high probability to the block arm for smoke/beta, but the feature vector provides no direct signal that smoke/beta is a high-failure project. In a real setting, project metadata (team size, language, test coverage) could make the failure rate more directly observable. The synthetic evaluation therefore measures a lower bound on bandit performance: a policy with access to project-type features could converge faster. The aggregate tie on the synthetic dataset does not reflect what would happen with a richer feature representation.

7.11 Bootstrap Results Depend on a Single Fixed Seed; Different Seeds May Yield Different p-Values

All bootstrap confidence intervals and paired bootstrap p-values in this paper use seed 42 with 10,000 resamples (as noted in §2 and `experiments/run_robustness.py`). The bootstrap procedure is deterministic given this seed. We did not evaluate sensitivity of p-values or CI endpoints to the choice of bootstrap seed. For the F9 Thompson-vs-LinUCB result ($p<0.0001$, 0/10,000 bootstrap samples with Thompson mean $\geq$ LinUCB), the finding is robust: the p-value is at the floor for any 10,000-resample procedure and will not change under different seeds. For findings where the p-value is near a threshold — particularly the Thompson-vs-static "tied" result, where static=644.5 falls inside CI [598, 648] — a different bootstrap seed could shift the CI endpoints by a few units and change the coverage interpretation. Readers should treat the [598, 648] endpoints as approximate to ± 5 units under seed variation.

---

8. Conclusion

What Works

Cost-sensitive reward design is the most impactful engineering decision. Replacing the asymmetric cost matrix with binary (-1/0) feedback degrades performance by 31%. This gap exists because binary reward cannot distinguish a deploy failure (cost 10) from a block that correctly prevented a failure (cost 0.5) — both receive the same -1 signal for failures. The cost matrix makes the asymmetry explicit and trainable.

The bandit framing delivers when both conditions in §1.3 hold. At deploy_failure=20 (doubled), the bandit outperforms static rules by 19% by learning a block-heavy strategy the static rule cannot reach. At low block cost, it outperforms by 27%. The framing's advantage is conditional, not universal — it requires sufficient failure rate and sufficient context signal to justify the exploration cost.

Thompson Sampling's stochastic exploration is the right tool for short or uncertain trajectories. Its mean performance on real data (624 vs. 644.5 for static rules; n=30 seeds) is 3.2% lower on the point estimate, but the two policies are statistically tied (static=644.5 falls inside Thompson's CI [598, 648]). Its variance across seeds (±72) makes it the only policy for which uncertainty quantification is meaningful in the current setup.

What Doesn't Work (and Why)

LinUCB over-blocks in the low-failure regime with feature sparsity. On psf/requests (5.3% failure, zero commit-level features), the bandit cannot distinguish the project from pallets/flask and applies a one-size-fits-all block-heavy strategy. At 5.3% failure, deploy is optimal (0.53/step vs. block 1.92/step); LinUCB's 42.2% block rate costs 3.8% more than a static rule. This is not a flaw in LinUCB — it is a flaw in the application: the convergence condition (informative context, sufficient trajectory length) is violated.

Drift detection is destructive on stationary data at default threshold. PageHinkley at λ_PH = 50 fires 44 false alarms, each resetting learned weights, resulting in a model 27% more expensive than no-drift. The component needs calibration against non-stationary data before any performance claim can be made.

The aggregate synthetic tie is misleading. LinUCB and static rules tie at 1878 vs. 1879 only because the two synthetic projects are balanced to cancel. Project-level, the results diverge substantially (897 vs. 966 on the 35% project; 911 vs. 982 on the 15% project). Aggregates hide the structure.

What This Changes About Deployment Decision Research

The classification framing of JIT defect prediction (Kamei et al., 2013) encodes a fixed, implicit cost assumption at the threshold. When that assumption is wrong — when failure costs are high, when blocking is cheap, when the failure rate is project-specific — the threshold cannot adapt. A contextual bandit with an explicit cost matrix can. The cost matrix itself is the interface through which operational priorities are expressed; it should be reported as an explicit hyperparameter in any deployment control system, not buried in a threshold.

The negative result is equally informative: bandits do not uniformly outperform static rules. They require sufficient context signal and sufficient trajectory length. In the low-failure regime with feature sparsity, the exploration cost dominates and a well-tuned static rule is competitive. This defines the operating envelope for the framing.

Future Work

Three directions are tractable extensions of the current infrastructure:

1. **Feature-rich real data.** Fetch commit-level metadata (files changed, tests run, author history) alongside GitHub Actions run status. This populates all 12 non-bias dimensions of the feature vector and enables a fair comparison on real data.
2. **Non-stationary evaluation.** Design a synthetic environment with abrupt failure-rate shifts and evaluate whether PageHinkley (with calibrated λ_PH) reduces post-shift regret relative to the no-drift variant.
3. **Per-project exploration calibration.** Low-failure-rate projects benefit from lower α (less UCB exploration); high-failure projects benefit from higher α. A meta-policy that adapts α based on observed failure rate would avoid the over-blocking failure mode documented in §5.4.

---

9. Discussion

9.1 Why the Bandit Framing Wins (When It Does)

The cost-ratio sweep (F11, §6.4) makes the win condition precise: the bandit advantage over a static rule scales monotonically from -7% at 5:1 to -49.6% at 100:1, with the default 20:1 sitting at +0.05% — break-even. This is not a result that requires interpretation; it is the direct output of a mechanism. Cost-weighted updates assign gradient magnitude proportional to consequence. A deploy failure at 100:1 receives a weight update 100 times larger than blocking a safe commit. A static rule applies the same threshold regardless of what those errors cost. The sweep measures the compounding gap between adaptive and fixed behavior as the cost structure diverges from parity.

Condition 1 from §1.3 — sufficient failure-rate asymmetry — is what allows this mechanism to fire. When failure rates are low, the optimal action is nearly always deploy regardless of context, and exploration budget is wasted sampling suboptimal arms. When failure rates are high enough that the cost structure rewards discrimination, cost-weighted updates steer the policy toward block-heavy strategies the static rule cannot reach without manual recalibration. The 20:1 default is not in that regime. The 40:1 case — 18.9% advantage — is. The operating envelope boundary is not a conceptual claim; it is

a measured number specific to this synthetic setting, and it shifts with failure rate, trajectory length, and feature signal.

9.2 When Bandits Underperform

Three failure modes, each a distinct edge of the operating envelope.

Condition 1 violated: the low-failure-rate trap. On psf/requests (5.3% failure rate, F8), the optimal action is almost always deploy — expected cost is 0.53/step versus 1.92/step for block. A bandit with exploration budget allocates 42.2% of decisions to block before its posterior settles. That exploration is not recovered: the project's failure rate is too low to reward a block-heavy strategy. LinUCB ends 3.8% worse than a static rule that simply deploys aggressively. The failure is predictable from the framework: Condition 1 requires sufficient failure-rate asymmetry to justify exploration. At 5.3%, it is not there.

Condition 2 violated: the convergence floor. With d=13 features and 300 steps per project, each arm receives approximately 100 updates — below the $O(d^2) = 169$ threshold from §1.3 Condition 2. UCB confidence intervals do not shrink to first-order accuracy. The action distribution reflects initialization noise as much as context signal. A static rule encoding the right failure rate directly outperforms a bandit that cannot yet read the features it was given.

Miscalibrated drift detection: false alarms as the dominant cost. PageHinkley at λ=50 treats cost-stream variance as a drift signal. On stationary data it fires 10.6 resets per 500-step trajectory; under abrupt drift, 25.3; under gradual drift, 42.2 (F12). In all three modes, the full drift-adaptive variant is 8–28% more expensive than plain LinUCB. The resets destroy accumulated learning faster than post-drift adaptation compounds. This is a calibration failure — the threshold was never tuned to the cost stream's variance — not a fundamental argument against drift detection.

9.3 Practical Deployment Considerations

Three calibration decisions must be made before this framing is deployable. None of them have safe defaults.

Cost matrix. The 20:1 ratio used throughout this paper is not a recommendation — it is a break-even point. An internal tool and a payment-critical service operate at different effective cost ratios, and the matrix is the mechanism through which that difference enters the policy. The cost matrix is the most consequential hyperparameter in the system, more so than α or λ_PH, because it determines which part of the operating envelope the policy operates in. Setting it from a default rather than from the operational cost structure of the specific service means the results in this paper say nothing about whether the approach will work.

λ_PH calibration. The 10.6 false alarms per trajectory on stationary data at λ=50 are not a property of PageHinkley; they are a property of λ=50 applied to this cost stream's variance. A project-specific sweep over λ_PH — measuring false-alarm rate on a representative sample of the project's stationary cost data before enabling drift-triggered resets — is the prerequisite step this paper did not take. The sweep is cheap. Running the system without it, as this paper did, produced a component that was net-negative in every drift condition tested.

Cold-start. An untrained bandit in a low-failure-rate environment replicates the psf/requests failure mode from the first step. Every CD pipeline already generates logged-action data — sequences of (context, action, outcome) from its existing static policy. Bootstrapping the bandit on that data before going live removes the cold-start over-blocking risk at no additional data-collection cost. Deploying without it is the most avoidable mistake the approach enables.

The framing is worth the engineering investment when the cost structure is calibrated, the trajectory is long enough to cross the Condition 2 threshold, and cold-start risk is managed. The algorithmic gains documented in this paper are conditional on all three.

AI Use Statement

Generative AI assistance was used to draft the Abstract and §9 Discussion under the author's direction. All experimental design, code, analysis, numerical claims, and final wording decisions are the author's. Drafted prose was reviewed for accuracy against the per-claim validity classification in Appendix A. All other sections were authored by Abishek Kumar Giri.

References

- Agrawal, S., Goyal, N. "Thompson Sampling for Contextual Bandits with Linear Payoffs." ICML 2013.
- Beller, M., et al. "TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research." MSR 2017.
- Bifet, A., Gavaldà, R. "Learning from Time-Changing Data with Adaptive Windowing." SIAM SDM 2007.
- Chu, W., et al. "Contextual Bandits with Linear Payoff Functions." AISTATS 2011.

- Gama, J., et al. "A Survey on Concept Drift Adaptation." ACM CSUR 2014.
- Joachims, T., Swaminathan, A., Schnabel, T. "Unbiased Learning-to-Rank with Biased Feedback." WSDM 2018.
- Joulani, P., et al. "Online Learning under Delayed Feedback." ICML 2013.
- Kamei, Y., et al. "A Large-Scale Empirical Study of Just-in-Time Quality Assurance." TSE 2013.
- Li, L., et al. "A Contextual-Bandit Approach to Personalized News Article Recommendation." WWW 2010.
- McIntosh, S., Kamei, Y. "Are Fix-Inducing Changes a Moving Target?" EMSE 2018.
- Mouss, H., et al. "Test of Page-Hinckley, an Approach for Fault Detection in an Agro-Alimentary Production System." MED 2004.
- Pike-Burke, C., et al. "Bandits with Delayed, Aggregated Anonymous Feedback." ICML 2018.
- Vernade, C., Cappé, O., Perchet, V. "Stochastic Bandit Models for Delayed Conversions." UAI 2017.

Appendix A: Validity Classification

Labels: - **Demonstrated** — mechanism verified; result is robust across n=30 seeds, multiple configurations, and/or follows directly from code/math. Unlikely to reverse with more data. - **Suggestive** — result is real on the data collected, but scope is limited (synthetic, 2 projects, short trajectory). Directionally plausible; magnitude may shift. - **Speculative** — claim extrapolates beyond the experimental evidence, relies on analogy, or is a conditional prediction not yet tested.

Appendix B: Experiment Reproducibility

```
# Main online replay (includes Thompson Sampling)
for seed in 0 1 2 3 4; do
    python -m experiments.run_bandits \
        --config experiments/configs/online_smoke.json \
        --seed $seed
done

# Real-data sanity check (requires data/raw/github_actions_real.csv)
for seed in 0 1 2 3 4; do
    python -m experiments.run_bandits \
        --config experiments/configs/real_github_actions.json \
        --seed $seed
done

# Robustness sweep
python -m experiments.run_robustness \
    --configs experiments/configs/online_smoke.json \
              experiments/configs/robustness_high_failur
              experiments/configs/robustness_low_block.j
              experiments/configs/robustness_short_delay
              experiments/configs/robustness_long_delay.
    --seeds 0 1 2 3 4

# Ablation study
for seed in 0 1 2 3 4; do
    python -m experiments.run_ablations --seed $seed
done
```

Results are written to `experiments/results/` (gitignored). The smoke dataset is at `data/raw/travistorrent_smoke.csv` (gitignored). All policy implementations, evaluation runners, and config files are tracked in version control.

#	Claim	Section
1	The pending-reward buffer enforces the delayed-feedback invariant (no future information at decision time)	§2.2, F2
2	Removing the buffer improves cost by 1.1% on synthetic data	§5.3, F2
3	PageHinkley at λ_PH=50 fires 44 false alarms on 1,150 stationary steps	§5.3, F3
4	`linucb_with_drift_no_reset` is numerically identical to `linucb` in all three drift modes	§7.4, F12
5	Thompson Sampling produces nonzero seed variance (std=54 synthetic, std=72 real); LinUCB does not	§5.1, F4
6	Thompson's 95% bootstrap CI [598, 648] does not overlap LinUCB=669.5; $p<0.0001$ paired bootstrap	§5.4, F9
7	Real GitHub Actions data contains 17–22 censored rewards per trajectory ($\approx$3%)	§5.4, F10
8	Cost weighting is the dominant ablation component: binary reward degrades cost by +31%	§5.3, F1; Abstract (+31%)
9	LinUCB over-blocks in the low-failure regime and costs 3.8% more than static rules	§5.4, F8; Abstract (3.8%)
10	Thompson Sampling outperforms LinUCB by 6.8% on real data (624 vs 669.5)	§5.4, F9; Abstract (6.8%)
11	Thompson and the static rule are statistically tied on real data (static=644.5 inside CI [598, 648])	§5.4, F9
12	Bandit advantage scales with failure cost: +19% at 2× failure cost, +27% at lower block cost	§5.2, F5; Abstract (19%, 27%
13	Cost-ratio sweep: LinUCB advantage monotone from -7% (5:1) to -49.6% (100:1)	§6.4, F11; Abstract (-7%, -4
14	Severe delay (120s) costs 1.0% more than default; short delay (30s) reduces cost by 1.5%	§5.2, F6
15	The aggregate bandit/static tie (1878 vs 1879) is a per-project cancellation artifact	§5.1, F7
16	Thompson's posterior never fully collapses to the optimal arm (persistent canary allocation)	§1.3, §5.1
17	LinUCB advantage over static rules requires $\geq$40:1 cost ratio in this synthetic setting	§6.4, operating envelope
18	Page-Hinkley drift resets at λ=50 are net-negative across all three drift modes	§6.4, F12
19	The 5.3% failure rate of psf/requests is a measured data property	§5.4; Abstract (5.3%)
20	The bandit framing helps when failure costs are high and context is informative	§1.2, Abstract
21	Per-project exploration calibration (adapting α to observed failure rate) would avoid over-blocking	§9
22	Longer trajectories would amortize reset overhead under drift	§6.4
23	The 20:1 default cost ratio sits at the operating envelope boundary where bandits break even	§1.4, Abstract